

Levantar el backend desde cero

Guía para montar el proyecto en una máquina nueva. Pensada para Windows con PowerShell, que es lo que usa el equipo.

Tiempo estimado: 20 minutos, casi todos esperando descargas.

Antes de empezar: instalar esto

Programa	Para qué	Dónde
Node.js 20 o superior	correr el backend	https://nodejs.org (versión LTS)
Docker Desktop	la base de datos	https://www.docker.com/products/docker-desktop
Git	clonar el repositorio	https://git-scm.com
pgAdmin (opcional)	ver la base con interfaz gráfica	https://www.pgadmin.org/download/

Comprobar que quedaron bien instalados:

```
node -v      # v20.x.x o superior
docker -v    # Docker version 2x.x.x
git --version
```

Docker Desktop tiene que estar ABIERTO y corriendo antes del paso 2. Si el icono de la ballena en la barra de tareas no está en verde, los comandos `docker` van a fallar con *"error during connect"*.

No hace falta instalar PostgreSQL. Va dentro de Docker. Si ya lo tenés instalado en Windows, no molesta: por eso usamos el puerto 5433.

pgAdmin es opcional y se puede dejar para después: en el **Paso 7** están las tres formas de conectarse, y la primera no requiere instalar nada.

Paso 1 — Clonar e instalar dependencias

```
git clone https://github.com/MejiaEduardo/backend-Odonto_Bases.git
cd backend-Odonto_Bases
npm install
```

Todo vive en la raíz del repositorio: el `package.json`, el `docker-compose.yml` y los archivos `.sql`. No hay carpetas anidadas.

`npm install` tarda unos minutos y suele mostrar avisos amarillos de vulnerabilidades: son de dependencias indirectas y no impiden trabajar.

Paso 2 — Levantar la base de datos

Desde la misma carpeta del repositorio:

```
docker compose up -d
```

Comprobar que arrancó:

```
docker compose ps
```

Tiene que aparecer `odonto-db` con estado `running (healthy)`. Si dice `starting`, esperá unos segundos y repetí.

Paso 3 — Cargar el esquema y los datos

El orden importa. Cada script asume que el anterior ya corrió.

Por qué `docker cp` y no `psql < archivo.sql`: en PowerShell el operador `<` no existe y da *ParserError*. Además, canalizar el archivo corrompe los acentos. Copiar el archivo dentro del contenedor y ejecutarlo con `-f` evita las dos cosas.

Copiar todos los `.sql` de una vez:

```
docker cp tablas.sql           odonto-db:/tmp/
docker cp datos.sql            odonto-db:/tmp/
docker cp arreglar-updatedAt.sql odonto-db:/tmp/
docker cp arreglar-passwords.sql odonto-db:/tmp/
docker cp migracion-estado-solicitada.sql odonto-db:/tmp/
docker cp migracion-servicios-odontologicos.sql odonto-db:/tmp/
docker cp migracion-aviso-cancelacion.sql odonto-db:/tmp/
docker cp migracion-evitar-duplicados.sql odonto-db:/tmp/
```

Y ejecutarlos **en este orden exacto**:

```
# 1. Estructura: tablas, tipos ENUM, llaves foráneas
docker exec odonto-db psql -U postgres -d odontologia -f /tmp/tablas.sql

# 2. Poner DEFAULT a las columnas updatedAt.
#   Va ANTES de los datos: 10 tablas tienen "updatedAt" NOT NULL sin
#   DEFAULT (residuo del esquema estilo Prisma) y cualquier INSERT que
#   no la rellene explícitamente falla.
docker exec odonto-db psql -U postgres -d odontologia -f /tmp/arreglar-updatedAt.sql

# 3. Datos de ejemplo (personas, empleados, citas...)
docker exec odonto-db psql -U postgres -d odontologia -f /tmp/datos.sql

# 4. Contraseñas de verdad.
#   En datos.sql son marcadores tipo '$2b$10$hashpassword01': parecen
#   hashes de bcrypt pero miden 21 caracteres y uno real mide 60.
#   Sin este paso NADIE puede iniciar sesión.
docker exec odonto-db psql -U postgres -d odontologia -f /tmp/arreglar-passwords.sql

# 5. Estado SOLICITADA (el cliente pide, la clínica aprueba)
docker exec odonto-db psql -U postgres -d odontologia -f /tmp/migracion-estado-solicitada.sql

# 6. Catálogo odontológico
#   Los datos de ejemplo eran de una clínica médica general
#   (Cardiología, Electrocardiograma...). Esto los reemplaza por
#   Ortodoncia, Endodoncia, Periodoncia, etc.
docker exec odonto-db psql -U postgres -d odontologia -f /tmp/migracion-servicios-odontologicos.sql

# 7. Aviso de cancelación al paciente
docker exec odonto-db psql -U postgres -d odontologia -f /tmp/migracion-aviso-cancelacion.sql

# 8. Índice único sobre el DNI (evita pacientes duplicados)
docker exec odonto-db psql -U postgres -d odontologia -f /tmp/migracion-evitar-duplicados.sql
```

Todos son **idempotentes**: si algo sale mal se pueden repetir sin romper nada.

NO corran `limpiar-duplicado-david.sql`. Ese script borra una persona concreta (id 22) de la base de Fernando. En una instalación nueva no existe y no hace falta.

Comprobar que quedó bien

```
docker exec odonto-db psql -U postgres -d odontologia -c "SELECT COUNT(*) AS personas FROM
\"Persona\";"
docker exec odonto-db psql -U postgres -d odontologia -c "SELECT id, nombre FROM
\"ServicioClinico\" ORDER BY id LIMIT 5;"
```

Los servicios que salgan tienen que ser dentales. Si aparece "Electrocardiograma", faltó el paso 6.

Paso 4 — Crear el archivo `.env`

Copiar la plantilla:

```
copy .env.example .env
```

Los valores por defecto ya coinciden con `docker-compose.yml`, así que funciona tal cual. Lo único recomendable es cambiar `JWT_SECRET`.

Contenido mínimo que tiene que quedar:

```
DB_HOST=localhost
DB_PORT=5433
DB_USER=postgres
DB_PASSWORD=odonto123
DB_NAME=odontologia
PORT=3000
JWT_SECRET=cualquier-cadena-larga-y-aleatoria
```

DB_PORT es 5433, no 5432. El 5432 lo ocupa el PostgreSQL instalado en Windows, que es una base vacía y distinta. Este error ya nos costó una tarde: pgAdmin mostraba la base sin datos y creímos que se había corrompido, cuando simplemente estábamos mirando el servidor equivocado.

No existe DATABASE_URL. Si ven esa variable en algún apunte viejo, ignórenla: el código lee las cinco `DB_*` de arriba.

Google y Firebase son **opcionales**: sin ellos el backend arranca igual. Google solo hace que no aparezca ese botón en el login; Firebase solo afecta la subida de archivos adjuntos.

Paso 5 — Arrancar

```
npm run start:dev
```

Cuando esté listo se ve algo así:

```
[Nest] LOG [NestApplication] Nest application successfully started
```

Comprobar que responde:

```
curl.exe http://localhost:3000/Servicios
```

Tiene que devolver el JSON con los servicios dentales.

En PowerShell, `curl` es un alias de `Invoke-WebRequest` y se comporta distinto. Hay que escribir **`curl.exe`**, con la extensión.

La documentación interactiva queda en <http://localhost:3000/api> (Swagger), donde se pueden probar todos los endpoints.

Paso 6 — Iniciar sesión

Después de `arreglar-passwords.sql`, los usuarios de prueba quedan con contraseñas conocidas. El script las imprime al final; pedíselas a Fernando o miralas ahí.

Roles disponibles: **ADMIN**, **DOCTOR**, **RECEPCIONISTA** y **CLIENTE**. Cada uno entra a una pantalla distinta.

Paso 7 — Conectarse a la base de datos

Aquí es donde se hacen las consultas y se comprueba que todo esté guardando bien. Hay tres formas; **la primera no requiere instalar nada**.

Datos de conexión (los mismos para las tres)

Campo	Valor
Host / Servidor	localhost
Puerto	5433
Base de datos	odontologia
Usuario	postgres
Contraseña	odonto123

El error del puerto — leer esto antes de conectarse

El puerto es **5433**, no el 5432 de siempre.

Docker publica el contenedor en el 5433 justamente porque el 5432 suele estar ocupado por un PostgreSQL instalado en Windows. Si se conectan al 5432 **la conexión funciona igual**, pero entran a otro servidor: una base vacía y distinta.

A nosotros nos pasó: pgAdmin mostraba las tablas sin datos y los acentos raros, y creímos que la base se había corrompido. Estuvimos a punto de borrarla y volver a cargar todo. No había nada roto — sólo estábamos mirando el servidor equivocado.

Si ven la base vacía, lo primero que hay que revisar es el puerto.

Opción A — psql dentro del contenedor (sin instalar nada)

Es la más rápida y la que usamos para los scripts. El cliente `psql` ya viene dentro de la imagen de Docker.

Consulta suelta, sin entrar a la consola:

```
docker exec odonto-db psql -U postgres -d odontologia -c "SELECT COUNT(*) FROM \"Persona\";"
```

Sesión interactiva, para varias consultas seguidas:

```
docker exec -it odonto-db psql -U postgres -d odontologia
```

El prompt cambia a `odontologia=#`. A partir de ahí se escribe SQL normal, terminando **siempre con punto y coma**:

```
SELECT id, nombre, apellido FROM "Persona" LIMIT 5;
```

Si el prompt cambia a `odontologia-#` (guion en vez de almohadilla) es que falta el punto y coma: `psql` sigue esperando el resto de la instrucción. Escriban `;` y Enter.

Comandos útiles de psql (empiezan con barra invertida, sin punto y coma):

Comando	Qué hace
<code>\dt</code>	listar todas las tablas
<code>\d "Persona"</code>	ver las columnas y los índices de una tabla
<code>\du</code>	listar los usuarios de la base
<code>\l</code>	listar las bases de datos
<code>\x</code>	alternar vista vertical (útil con muchas columnas)
<code>\timing</code>	mostrar cuánto tarda cada consulta
<code>\q</code>	salir

Ejecutar un archivo .sql: primero copiarlo dentro del contenedor y después ejecutarlo con `-f` (nunca con `<`, que en PowerShell no existe):

```
docker cp mi-consulta.sql odonto-db:/tmp/
docker exec odonto-db psql -U postgres -d odontologia -f /tmp/mi-consulta.sql
```

Opción B — pgAdmin (interfaz gráfica oficial)

Más cómodo para explorar tablas y ver resultados en cuadrícula.

- 1. Instalar.** Descargar de <https://www.pgadmin.org/download/> e instalar con las opciones por defecto. La primera vez pide crear una *contraseña maestra*: es solo de pgAdmin, no tiene nada que ver con la base. Anótenla.
- 2. Registrar el servidor.** Clic derecho sobre **Servers** en el panel izquierdo → **Register** → **Server...**
- 3. Pestaña General:**
 - **Name:** `Odonto Docker` (es solo una etiqueta, pongan lo que quieran)
- 4. Pestaña Connection** — aquí es donde se equivoca todo el mundo:

Campo	Valor
Host name/address	localhost
Port	5433 ← cambiarlo, viene 5432 por defecto
Maintenance database	odontologia
Username	postgres
Password	odonto123
Save password?	activado

5. Clic en **Save**. Si todo está bien, el servidor aparece en el panel izquierdo y se puede desplegar.

6. **Encontrar las tablas.** Están más adentro de lo que uno espera:

```

Servers
├── Odonto Docker
│   ├── Databases
│   │   ├── odontologia
│   │   │   ├── Schemas
│   │   │   │   ├── public
│   │   │   │   │   └── Tables ← aquí están

```

7. **Ver el contenido de una tabla:** clic derecho sobre la tabla → *View/Edit Data* → *All Rows*.

8. **Escribir consultas:** seleccionar la base `odontologia` y abrir **Tools → Query Tool** (o el icono del rayo). Escribir el SQL y pulsar **F5** para ejecutar.

Opción C — DBeaver (alternativa más liviana)

Si pgAdmin les resulta pesado. Descargar la *Community Edition* de <https://dbeaver.io/download/>.

1. **Nueva conexión** (icono del enchufe con un **+**) → elegir **PostgreSQL** → *Siguiente*.
2. Rellenar: Host `localhost`, **Port** `5433`, Database `odontologia`, Usuario `postgres`, Contraseña `odonto123`.
3. La primera vez ofrece **descargar el driver** de PostgreSQL: aceptar.
4. **Test Connection** para comprobar, y después *Finalizar*.

Las comillas dobles son obligatorias

Esto confunde a todo el mundo la primera vez:

```

SELECT * FROM Persona;      -- ERROR: relation "persona" does not exist
SELECT * FROM "Persona";    -- correcto

```

PostgreSQL convierte a minúsculas cualquier identificador que no vaya entre comillas. Como las tablas de este proyecto se crearon con mayúsculas (`"Persona"`, `"User"`, `"ServicioClinico"`), hay que escribirlas **siempre entre comillas dobles** y respetando las mayúsculas.

Lo mismo aplica a las columnas en *camelCase*: "pacienteId", "updatedAt", "numeroFactura".

Desde PowerShell, con `psql -c "..."`, las comillas dobles internas hay que escaparlas con barra invertida: `-c "SELECT * FROM \"Persona\";"`. Dentro de una sesión interactiva o de un archivo `.sql` no hace falta.

Consultas para revisar que todo funciona

Sirven para verificar el estado del sistema sin pasar por la interfaz.

¿Se cargaron los datos?

```
SELECT
  (SELECT COUNT(*) FROM "Persona")          AS personas,
  (SELECT COUNT(*) FROM "User")              AS usuarios,
  (SELECT COUNT(*) FROM "Empleado")          AS empleados,
  (SELECT COUNT(*) FROM "Cita")              AS citas,
  (SELECT COUNT(*) FROM "Expediente")        AS expedientes,
  (SELECT COUNT(*) FROM "ExpedienteDetalle") AS consultas,
  (SELECT COUNT(*) FROM "Factura")           AS facturas;
```

¿El catálogo es odontológico? No debe aparecer nada tipo "Electrocardiograma":

```
SELECT id, nombre, precio, activo
FROM "ServicioClinico"
ORDER BY id;
```

¿Con qué usuarios se puede entrar?

```
SELECT u.correo, u.rol, p.nombre || ' ' || p.apellido AS persona
FROM "User" u
JOIN "Persona" p ON p.id = u."personaId"
ORDER BY u.rol, u.correo;
```

¿Las contraseñas son válidas? Un hash real de bcrypt mide exactamente 60 caracteres. Si sale algo de 21, faltó correr `arreglar-passwords.sql`:

```
SELECT correo, LENGTH(password) AS largo_hash
FROM "User"
ORDER BY largo_hash;
```

Citas por estado — para entender en qué punto del flujo está cada una:

```
SELECT estado, COUNT(*) AS cantidad
FROM "Cita"
GROUP BY estado
ORDER BY cantidad DESC;
```

¿Hay citas listas para facturar? Son las COMPLETADA que todavía no tienen factura; es lo que alimenta la pantalla *Generar Factura*:


```

SELECT c.id, c.fecha, p.nombre || ' ' || p.apellido AS paciente, s.nombre AS servicio
FROM "Cita" c
JOIN "Persona" p      ON p.id = c."pacienteId"
JOIN "ServicioClinico" s ON s.id = c."servicioId"
WHERE c.estado = 'COMPLETADA'
      AND NOT EXISTS (SELECT 1 FROM "Factura" f WHERE f."citaId" = c.id)
ORDER BY c.fecha DESC;

```

Agenda de un doctor:

```

SELECT c.fecha, c.hora, c.estado,
       pac.nombre || ' ' || pac.apellido AS paciente,
       s.nombre AS servicio
FROM "Cita" c
JOIN "Persona" pac      ON pac.id = c."pacienteId"
JOIN "ServicioClinico" s ON s.id = c."servicioId"
WHERE c."doctorId" = 1      -- cambiar por el id del empleado
ORDER BY c.fecha DESC, c.hora;

```

¿Se guardó la consulta que acabo de registrar? Útil para comprobar que el expediente se actualizó de verdad:

```

SELECT d.id, d.fecha, p.nombre || ' ' || p.apellido AS paciente,
       d.motivo, d.diagnostico
FROM "ExpedienteDetalle" d
JOIN "Expediente" e ON e.id = d."expedienteId"
JOIN "Persona" p    ON p.id = e."pacienteId"
ORDER BY d.id DESC
LIMIT 10;

```

¿Hay pacientes duplicados?

```

SELECT LOWER(TRIM(nombre)) || ' ' || LOWER(TRIM(apellido)) AS persona,
       COUNT(*) AS veces, array_agg(id ORDER BY id) AS ids
FROM "Persona"
GROUP BY 1
HAVING COUNT(*) > 1;

```

Ver la estructura de una tabla (columnas, tipos y si aceptan nulos):

```

SELECT column_name, data_type, is_nullable, column_default
FROM information_schema.columns
WHERE table_name = 'Persona'
ORDER BY ordinal_position;

```

Si la conexión falla

Mensaje	Causa	Solución
Connection refused / could not connect to server	el contenedor no está corriendo	<code>docker compose up -d</code>
Conecta pero no aparece la base <code>odontologia</code>	están en el 5432 (PostgreSQL de Windows)	cambiar el puerto a 5433
La base aparece vacía	mismo caso que el anterior	revisar el puerto
password authentication failed for user "postgres"	contraseña incorrecta	es <code>odonto123</code> , la de <code>docker-compose.yml</code>
database "odontologia" does not exist	faltó cargar el esquema, o puerto equivocado	revisar el puerto y después el Paso 3
relation "persona" does not exist	falta escribir las comillas dobles	usar <code>"Persona"</code> , con mayúscula inicial
syntax error at end of input	falta el punto y coma	agregar <code>;</code> y Enter

Comprobar rápido a qué servidor están conectados, si hay dudas:

```
SELECT current_database(), inet_server_port(), version();
```

Tiene que decir `odontologia` y el puerto interno `5432` — que es correcto: **dentro** del contenedor PostgreSQL usa el 5432; el 5433 es solo la puerta de entrada desde Windows.

Uso diario

Una vez montado, arrancar cada día es solo esto:

```
# 1. Abrir Docker Desktop y esperar a que el icono esté en verde
# 2. Base de datos
cd C:\ruta\a\backend-Odonto_Bases
docker compose up -d

# 3. Backend (misma carpeta)
npm run start:dev
```

Apagar al terminar (**los datos se conservan**):

```
docker compose down
```

Si algo falla

Síntoma	Qué pasa	Solución
<code>error during connect ...</code> <code>docker_engine</code>	Docker Desktop está cerrado	abrirlo y esperar a que el icono esté verde
<code>ECONNREFUSED 127.0.0.1:5433</code>	el contenedor no está corriendo	<code>docker compose up -d</code>
<code>password authentication failed</code>	el <code>.env</code> no coincide con <code>docker-compose.yml</code>	revisar <code>DB_USER</code> y <code>DB_PASSWORD</code>
El backend arranca pero toda consulta falla	están apuntando al puerto 5432	poner <code>DB_PORT=5433</code>
<code>Credenciales Invalidas</code> con la contraseña correcta	faltó <code>arreglar-passwords.sql</code>	correr el paso 3.4
<code>relation "Persona" does not exist</code>	faltó <code>tablas.sql</code>	correr el paso 3 desde el principio
<code>null value in column "updatedAt"</code>	faltó <code>arreglar-updatedAt.sql</code>	correr el paso 3.2
Aparecen servicios médicos, no dentales	faltó la migración del catálogo	correr el paso 3.6
<code>Unknown authentication strategy 'google'</code>	no debería pasar ya	ver <code>GUIA_LOGIN_GOOGLE.md</code>
<code>ParserError</code> al correr SQL	usaron <code>psql < archivo.sql</code>	usar <code>docker cp + psql -f</code>
Avisos de Firebase en la consola	no hay credenciales configuradas	es normal, solo afecta archivos adjuntos

Empezar la base completamente de cero

Si la base quedó en un estado raro y quieren borrarla y rehacerla:

```
docker compose down -v      # el -v BORRA los datos, no hay vuelta atrás
docker compose up -d
# y repetir el Paso 3 completo
```

Para tener en cuenta al trabajar

- **No se usa ningún ORM.** Los nombres de tabla en PascalCase entre comillas (`"Persona"` , `"User"`) son residuo de un esquema generado con Prisma, pero todo es SQL plano con la librería `pg` . Las comillas dobles son obligatorias: sin ellas PostgreSQL pasa el nombre a minúsculas y no encuentra la tabla.
- **Es PostgreSQL, no SQL Server.** SQL Server Management Studio no sirve. Para conectarse y consultar la base, ver el **Paso 7**.

- El `.env` no se sube al repositorio (está en `.gitignore`). Cada quien crea el suyo desde `.env.example`.
- Antes de subir cambios, comprobar que compila: `powershell npx tsc --noEmit -p tsconfig.json` # tiene que dar 0 errores

Pendientes conocidos

Estas tres cosas **todavía no funcionan**. Están anotadas para que nadie pierda tiempo pensando que las instaló mal.

1. Subir archivos a los expedientes — falta que Hugo comparta las credenciales

Los adjuntos se guardan en Firebase Storage, y hacen falta dos variables que **no están en el repositorio**:

```
FIREBASE_SERVICE_ACCOUNT_KEY= # el JSON de la cuenta de servicio, en UNA sola línea
FIREBASE_STORAGE_BUCKET=      # nombre-del-bucket.appspot.com
```

Hugo las tiene y debe pasarlas al equipo. Hasta entonces el backend arranca igual y avisa por consola; lo único que falla es adjuntar archivos. El resto del sistema no se ve afectado.

2. Los expedientes no se actualizan

Al guardar una consulta no siempre queda registrada. El caso concreto es el de **David Lopex**: se guardó y el expediente siguió vacío.

Ya se corrigió una causa —el `doctorId` estaba fijo en `1` en `ExpedienteDetails.tsx` en vez de tomarse de la sesión— y ahora el formulario muestra el mensaje real del backend en lugar de un "Error al guardar el detalle" genérico. **Falta comprobar si con eso alcanza.**

3. Reportes de administración no incluye a los doctores nuevos

La pestaña de reportes no se actualiza cuando se agrega un doctor: sigue mostrando solamente los que ya estaban. Hay que revisar la consulta de `GET /facturas/reportes`, que probablemente une contra los doctores que tienen facturas en lugar de listarlos todos.

Otros documentos del proyecto

Archivo	Para qué
<code>ESTADO_ACTUAL.md</code>	en qué quedó el trabajo y qué falta
<code>GUIA_BACKEND.md</code>	cómo está organizado el backend
<code>GUIA_LOGIN_GOOGLE.md</code>	activar el login con Google
<code>PRUEBAS_PERMISOS.md</code>	verificar permisos por rol